

Operator Decision Brief — Wiring ota-manager's Feature Pages (Router Reconciliation)

Revision: 1 **Last modified:** 2026-07-09T20:16:02Z **Authority:** Stream OB2 (autonomous research/decision-brief subagent) **Status:** PROPOSAL ONLY — un-executed, OPERATOR-GATED (§11.4.101 item C). No code changed. **Scope:** clients/ota-manager (React 19 + Tauri v2). Read-only investigation + deep-web citation. **Honest boundary (§11.4.6):** This document proves the CURRENT state as fact and lays out options. It recommends but does NOT execute. Wiring the pages is a UX + architecture decision the operator must approve. Nothing here has been implemented.

1. Current state — established as FACT (cite file:line)

1.1 Router library actually in use

- **App router** = `@tanstack/react-router@1.170.16` (resolved via `pnpm ls`).
 - `src/main.tsx:4-6,26` — `createRouter({ routeTree })` from `./route-tree.gen`, wrapped in `<RouterProvider>`.
 - `src/App.tsx:1,8` — a SECOND, redundant `createRouter({ routeTree })` (an alternate entry; `main.tsx` is the one Vite loads via `index.html`).
- **react-router-dom@7.18.0** is **ALSO a dependency** (`package.json:34`) but is a LEFTOVER — it is imported by exactly ONE non-test source file (see 1.4). It is not the app's router.

1.2 What is ACTUALLY wired into the router

The route tree is **hand-written**, NOT file-based codegen, at `src/route-tree.gen.ts`. Despite the `.gen.ts` name it is a normal committed source file using the code-based `createRoute` API. It wires only THREE things:

- `src/route-tree.gen.ts:12-16` — `/login` → `LoginPage`
- `src/route-tree.gen.ts:18-22` — `layout route` → `MainLayout` (`src/features/layout/main-layout.tsx`)
- `src/route-tree.gen.ts:24-28` — `/dashboard` → `DashboardPage` imported **from** `@/routes/dashboard` (the STATIC placeholder, see 1.5)

So the shipped app exposes **only** `/login` and a placeholder `/dashboard`. Everything else is unrouted.

1.3 Pages that EXIST under `src/features/**` (8 feature pages + 2 detail pages)

Page file	 Routed?	tsc errors
<code>src/features/auth/login-page.tsx</code>	wired (<code>/login</code>)	0
<code>src/routes/dashboard.tsx</code> (static placeholder)	 wired (<code>/dashboard</code>)	0
<code>src/features/dashboard/dashboard-page.tsx</code> (rich)	 UNROUTED	1
<code>src/features/devices/devices-page.tsx</code>	 UNROUTED	9
<code>src/features/devices/device-detail-page.tsx</code>	 UNROUTED	12
<code>src/features/releases/releases-page.tsx</code>	 UNROUTED	15
<code>src/features/deployments/deployments-page.tsx</code>	 UNROUTED	11
<code>src/features/deployments/</code>	UNROUTED	14

Page file	^x Routed?	tsc errors
deployment-detail-page.tsx	^x	
src/features/groups/groups-page.tsx	^x UNROUTED	6
src/features/audit/audit-page.tsx	UNROUTED	2
src/routes/index.tsx (route definitions, unused)	^x UNROUTED	10
src/features/layout/app-layout.tsx (dead — see 1.6)	UNROUTED	3
src/features/layout/sidebar.tsx	(rendered by MainLayout)	2

1.4 Exactly which files import which router

- **react-router-dom** (grep src/):
 - src/features/dashboard/dashboard-page.tsx:7 — import { useNavigate } from "react-router-dom";
 - src/__tests__/dashboard.test.tsx:6 — vi.mock('react-router-dom', ...) (test only)
- **@tanstack/react-router** (grep src/): App.tsx, main.tsx, route-tree.gen.ts, routes/index.tsx, routes/__root.tsx, features/deployments/deployments-page.tsx, features/deployments/deployment-detail-page.tsx, features/devices/device-detail-page.tsx, features/layout/{app-layout,main-layout,sidebar,topbar}.tsx, hooks/use-auth.ts, plus tests.

Net: only ONE production file (features/dashboard/dashboard-page.tsx) still uses the wrong router (react-router-dom). Most unrouted pages ALREADY import @tanstack/react-router but use its API incompletely / against a route graph that doesn't contain their paths.

1.5 Two competing dashboards (parallel-agent artifact)

- src/routes/dashboard.tsx — exports DashboardPage, a STATIC placeholder (“—” values, “Connect to server”). No hooks, no router API. This is the one currently wired.
- src/features/dashboard/dashboard-page.tsx — the RICH dashboard: telemetry stat cards (useTelemetryOverview), activity

feed (useAuditLog), quick-action buttons that navigate(...). Uses react-router-dom's useNavigate. Imported only by the unrouted routes/index.tsx + its test.

1.6 routes/index.tsx and app-layout.tsx are unreferenced

- src/routes/index.tsx uses createFileRoute("/dashboard") etc. — the **file-based** routing API, which requires the TanStack Router codegen plugin the project does not run. It is imported by **nothing** (route-tree.gen.ts is hand-written). Its createFileRoute calls produce 10 of the 85 errors (“not assignable to parameter of type undefined” — file-based routes need the generated register).
- src/features/layout/app-layout.tsx (AppLayout, default export) is imported by **nothing** — the wired layout is main-layout.tsx (MainLayout). app-layout.tsx is a dead duplicate.

1.7 Dead navigation today

src/features/layout/sidebar.tsx:18-23 renders nav links to / dashboard, /devices, /releases, /deployments, /groups, /audit. Only / dashboard resolves. The other five are **dead nav** — a user clicking them today hits a path with no route.

2. The 85 tsc errors — real count + categorized root cause

Real count: 85 errors (pnpm exec tsc --noEmit, TOTAL grep -c "error TS" = 85). Evidence captured at /tmp/tsc_out.txt during this run. Distribution by file matches §1.3.

CORRECTION to the session's framing. The 85 errors are ALL *located in* the unrouted-page cluster, but they are **NOT all router-API root cause**. They are a MIX of four independent drifts plus noise. Migrating react-router-dom → @tanstack/react-router **alone fixes only ~19 of 85 (~22%)**; wiring the pages requires reconciling three OTHER drift families too.

#	Root-cause family	~count	What it is	Fixed by router migration?
A		10		

#	Root-cause family	~count	What it is	Fixed by router migration?
	Router-API — file-based createFileRoute unused		routes/index.tsx uses file-based API with no codegen → “not assignable to undefined”	Yes (rewrite as code-based routes)
B	Router-API — navigate()/Link to/NavLink isActive/ ConstrainLiteral	~9	to="/deployments" not in route union; navigate("string") vs navigate({to}); NavLink className={{({isActive})=>...} (react-router signature) Pages expect the OLD custom-hook shape ({device, updates, loading, refresh}) but hooks now return react-query UseQueryResult ({data, isLoading, refetch}) — e.g. device-detail-page.tsx:460-472, devices-page.tsx:415, app-layout.tsx:89 (useLogout().mutate)	Yes
C	Hook-return-shape drift	~15	Property 'value'/'onChange' does not exist on '{ name: string }'; {placeholder} not assignable to — the Select/form primitives’ props changed	No — separate fix
D	Select/form component API drift	10	List-wrapper vs array (ReleaseList/DeploymentList/GroupList/AuditLogList/ Artifact used as arrays via .map/.find/.length/.filter); missing exports (Release, Deployment); wrong request fields (targetModel vs target_model, artifactId, RecallRequest.reason,	No — separate fix
E	API type-model drift	~22		No — separate fix

#	Root-cause family	~count	What it is	Fixed by router migration?
			DeviceListFilter.search, DeploymentStatus.id)	
F	Toast API drift	6	Property 'toast' does not exist on type 'ToastContextValue' — pages call toast() shape the context no longer exposes	No — separate fix
G	Unused imports (TS6133/TS6192)	8	Dead imports (React, SheetTrigger, CardDescription, groupId, deviceId, toast)	Incidental
H	Misc	~5	Type 'Error' is not assignable to type 'string' (error prop drift); Expected 1 arguments, but got 2/0	Partly

(Category boundaries A/B are router; C/D/E/F/G/H are not. Counts are ± 1 where an error line spans two families; the load-bearing conclusion — router migration $\approx 1/4$ of the work — is robust.)

Implication: “wire the pages” $\neq$ “migrate the router.” It is a **four-part reconciliation**: (1) router API + code-based route wiring, (2) hook-return-shape (adopt react-query {data, isLoading, refetch}), (3) API type-model (unwrap *List $\rightarrow$ arrays, fix request field names against the real Go server contract), (4) UI component props (Select/form, Toast). Only after all four does the unrouted cluster compile.

3. §11.4.124 investigate-before-remove — are these intended features or abandoned scaffolding?

Verdict: INTENDED SHIPPING FEATURES. Do NOT remove — wire them. Captured git + spec evidence:

- **Origin commit a0552d8e** “feat(ota-manager): cross-platform management client suite — 4 parallel agents” — the entire client (6,734 lines) was built in ONE deliberate pass by 4 parallel agents.

Agent #3's mandate is verbatim: *“Core UI Pages — Dashboard ... Devices ... Releases ... Deployments ... Groups ... Audit log ... All pages include loading skeleton, empty state, and error+retry.”* These pages are the product, not scaffolding.

- **Design spec docs/superpowers/specs/2026-06-19-ota-management-client-design.md §4 “Application Routes”** explicitly enumerates every route: `/devices`, `/devices/:id`, `/devices/:id/telemetry`, `/releases`, `/releases/:id`, `/deployments`, `/deployments/:id`, `/deployments/:id/rollout`, `/deployments/:id/recall`, `/groups`, `/groups/:id`, `/audit`. The component tree (§ lines 149-178) lists `DashboardPage/DevicesPage/ReleasesPage/DeploymentsPage/AuditPage` under the layout. This is a specified feature set.
- **The sidebar already links to them** (`sidebar.tsx:18-23`) — the nav was built expecting these routes.
- **They carry tests + host-render coverage** — `src/__tests__/features/{devices,releases,deployments}.test.tsx`, `src/__tests__/dashboard.test.tsx`, and `visual/run-all-dashboard.mjs` (§11.4.170 `host-render`). Abandoned scaffolding does not get a visual-regression harness.

Root cause of the “unrouted” state (git): the parallel-agent build produced TWO inconsistent routing approaches that were never reconciled — Agent #1 hand-wrote `route-tree.gen.ts` (code-based, login + placeholder dashboard only), while Agent #3 wrote the rich pages + a file-based `routes/index.tsx` that was never connected, and reached for `react-router-dom` habits in `dashboard-page.tsx`. Commit `34f7dcf6` (Stream P) later triaged 118 → 85 errors, explicitly deferring the remaining 85 as the “operator-gated tanstack-router reconciliation.” Commit `28ce6fd6` already applied §11.4.124/§11.4.114 to restore mistakenly-deleted hook shims in this same cluster — precedent that removal here is the wrong reflex.

The only genuine removal candidates (both superseded duplicates, still operator-gated per §11.4.122): - `src/features/layout/app-layout.tsx` — dead duplicate of `main-layout.tsx` (zero importers). - `src/routes/dashboard.tsx` static placeholder — to be RETIRED in favour of the rich `features/dashboard/dashboard-page.tsx` once wired (not deleted standalone). Neither should be removed except as part of the approved wiring work, each in its own §11.4.124 commit.

4. Options for the operator

Option	What it resolves	Effort	Risk	Needs UX decision?
A — WIRE (recommended): reconcile the 4 drift families (router + hooks + types + components) and add code-based routes for all pages into <code>route-tree.gen.ts</code>	All 85 → 0; ships the full specified management client; dead nav (§1.7) becomes live	High (multi-day; ~19 router + ~66 non-router fixes, each against the real Go server contract, §11.4.6)	Medium — bounded to clients/ota-manager; every page already has tests + host-render to guard the migration (§11.4.170)	Yes — confirm the v1 nav/route set + which sub-routes are in scope
B — DELETE the unrouted pages	All 85 vanish (errors leave with the files)	Low	Very high — DESTRUCTIVE. Removes intended, spec'd, tested features → §11.4.122 silent-removal violation + contradicts the design spec + throws away ~4,500 lines of built UI. Not recommended. Medium — tsc gate stays RED at 85 permanently; the product ships login + a “_” placeholder dashboard while a full	Yes (operator must explicitly approve dropping features)
C — LEAVE AS-IS	0	None		No

Option	What it resolves	Effort	Risk	Needs UX decision?
			client sits dark in the tree; dead nav misleads users	

Notes on Option A scope: the rich dashboard's quick-action buttons (dashboard-page.tsx) navigate to /releases/create, /deployments/create, /devices/register — sub-routes NOT in the design spec §4 list. The operator must decide whether those creation sub-routes are in the first wiring pass or the buttons are deferred/hidden.

5. Recommendation (OPERATOR-GATED — do NOT execute without approval)

Recommend Option A — wire the pages — as its own isolated feature work-stream (§11.4.167), because the pages are proven-intended, spec'd, and test-covered; the only alternative that clears the 85-error gate without destroying product is to build them, and they are already built — they need reconnecting, not writing.

Proposed concrete step list (each step its own reviewed commit; all gated behind operator go):

- 1. UX confirmation FIRST** (§11.4.66) — operator confirms the v1 route/nav set (proposed: the design spec §4 set, role-gated exactly as sidebar.tsx:18-23) and whether detail/creation sub-routes (/devices/:id, /deployments/:id, /releases/create, /deployments/create, /devices/register, telemetry/rollout/recall) are in-scope now or deferred.
- 2. Route wiring** — extend the hand-written src/route-tree.gen.ts with code-based createRoute children under the layout for each approved page (/devices, /releases, /deployments, /groups, /audit + approved detail routes). Retire the unused file-based src/routes/index.tsx.
- 3. Router-API migration (family A+B)** — replace react-router-dom useNavigate in dashboard-page.tsx with @tanstack/react-router; convert navigate("/x") → navigate({ to: "/x" }), Link to={"/x/\${id}`} → `Link to="/x/\${id}"`
params={{ id }}, useParams({ from }), and

NavLinkclassName={{isActive}}=>...}

→ TanStack active-props. Then dropreact-router-domfrompackage.json`.

4. **Hook-return-shape (family C)** — update pages to consume react-query { data, isLoading, refetch } instead of the legacy { device, loading, refresh }/useLogout().mutate shapes.
5. **API type-model (family E)** — unwrap *List wrappers to arrays where iterated, export Release/ Deployment, and fix request field names **against the real Go server wire contract** (read-only: server/internal/api/*wire.go — never modified, §11.4.6).
6. **Component-props (family D+F)** — reconcile Select/form value/ onChange/placeholder and the Toast context API; clean unused imports (G).
7. **Retire the two duplicates** — fold the rich dashboard in as / dashboard (retiring the routes/dashboard.tsx placeholder) and delete the dead features/layout/app-layout.tsx, each in its own §11.4.124 removal commit citing this brief.
8. **Prove it** — tsc 85 → 0; pnpm build exit 0; vitest green; host-render (visual/run-all*.mjs, §11.4.170) each screen × {light,dark}; then §11.4.185 manual-QA of live navigation.

UX decision the operator MUST make: *Which pages/routes ship in v1, and are the detail + creation sub-routes in scope now or deferred?* The default answer (design spec §4 + current sidebar, role-gated) is ready to approve as-is; the only open question is the creation sub-routes the dashboard quick-actions point at.

Sources verified

Verified 2026-07-09 against current TanStack Router documentation (v1.x) via Context7:

- TanStack Router — official migrate-from-react-router skill (packages/react-router/skills/lifecycle/migrate-from-react-router/SKILL.md): `Link to="/posts/$postId" params={{ postId }}` is the correct replacement for the react-router `to={` /posts/${postId}`}` habit.
- TanStack Router — `useNavigate()` (packages/react-router/skills/react-router/SKILL.md):
`navigate({ to: '/posts/$postId', params: { postId } })`.
- TanStack Router — `useParams({ from })` (same skill file).
- TanStack Router — code-based routing (docs/router/routing/code-based-routing.md): `createRoute({ getParentRoute, path,`

component }) — the API `route-tree.gen.ts` already uses and step 2 extends.